UNIVERSITY of the WESTERN CAPE

CSC312 GROUP ASSIGNMENT – 2026

Group name/number	***ByteFlow***
Date of submission	***2026/09/06***
Deliverable number & name	***Step 2: SE Group Project***
Tutor's name & period	***N/A***
Team leader's details	***4332981@myuwc.ac.za***

GROUP CONTRIBUTION MATRIX AND PLAGIARISM DECLARATION

#	Student number	Name	Surname	Signature	% contribution
1.	4332981	***CHRISTIAAN***	***GOUWS***	***C.G.***	***100***
2.	4368679	***JUDE***	***JOSHUA***		***100***
3.	4323227	***NOAH***	***MPHAHLELE***	***N.M***	***100***
4.	4101764	***NTSEKISENG ELLIOT***	***NYEMBE***	***N.E.***	***100***
5.	4388044	***REFILOE ANGELA***	***MSHIYO***	***R.M.***	***100***
6.	4395423	***YOLISA***	***MZWAKALI***		***100***

ASSIGNMENT SUBMISSION CHECKLIST

Google Document has been shared with tutor.
All team members have contributed to the document or % contribution has been indicated.
The link for the document has been pasted into the relevant submission section on Ikamva.
Full originality report (not digital receipt) is submitted.
Plagiarism declaration and AI declaration have been filled in and signed by all members.
and % contribution is signed and agreed upon by all members.
All relevant appendices have been added.
Detailed minutes of all group meetings, workload contribution/ delegation of tasks and roles, any other relevant/required documentation are included.
All instructions in the specification document have been met.
You have checked the rubric and meet all criteria.

PLAGIARISM DECLARATION

1. We know that plagiarism is wrong. Plagiarism is the use of another's work without acknowledging the origin of the work and pretending that it is one's own work.
2. We know that plagiarism is a punishable offence and that UWC has strict disciplinary measures against this.
3. We declare therefore, that each significant contribution to, and quotation in this document, that we have taken from the work(s) of other people have been correctly attributed and have been correctly cited (referenced) using the APA format.

4. We declare that this document is our own original work.
5. We have not allowed and will not allow anyone to copy our work with the intention of passing it off as his or her own work.

Software Engineering Practical Assignment
Project Proposal

by

(ByteFlow)

1. Project Abstract & Problem Statement....3
1.1. Problem Definition.... 3
1.2. Core Aim....3
2. Stakeholder Identification....3
2.1. Primary Stakeholders.... 3
2.2. Secondary Stakeholders....3
3. Software Process Model & Engineering Practices....4
3.1. Process Model....4
3.2. Process Elements....4
3.2.1. SE Principles....4
3.2.2. SE Methods/Techniques....4
3.2.3. SE Goals (ISO 25010)....4
3.2.4. SE Umbrella Activities....5
3.3. Toolchain....5
4. Project Scope & Requirements....5
4.1. Functional Requirements....5
4.2. Non-Functional Requirements....5

1. Project Abstract & Problem Statement

1.1. Problem Definition

The Max-Cut problem is one of the most classic problems in computer science. It's an NP-hard combinatorial optimisation challenge where a graph must be partitioned into two sets to maximise the weight of edges crossing between them. A problem arises when the graph grows; classical algorithms encounter exponential time complexity ($O(2^V)$), making optimal solutions computationally infeasible.

Quantum algorithms like Quantum Approximate Optimisation Algorithm (QAOA) already offer theoretical approaches to optimisation. But software developers and researchers lack the accessible visual tools to directly analyse and benchmark performance differences, execution times, and output accuracy between classical heuristics and real quantum cloud hardware.

1.2. Core Aim

Our aim is to develop a web-based application or a locally installed software that visualises custom graph topographies, and execute the Max-Cut algorithm using both classical methods and quantum methods (QAOA via IBM Quantum Cloud Runtime). The software will compare the performance metrics such as cut value accuracy, execution runtimes, and circuit depth.

2. Stakeholder Identification

2.1. Primary Stakeholders

- **Software developers:** The project aims to assist them in identifying the power of Quantum computing resources and how they can aid programmers in solving complex computations
- **Researchers:** Raising awareness of Quantum computing tools to researchers and showing the ease of accessibility, showing them that it can be a useful tool in furthering software development and engineering
- **IBM:** The use of their services for research or algorithms granting them an increase in net worth and potential profits.

2.2. Secondary Stakeholders

- **Consumers of technology** - Improvement in software development tools with the use of quantum computing can result in more efficient and robust software benefitting the consumer.

3. Software Process Model & Engineering Practices

3.1. Process Model

The team will use the Agile/Scrum software process. Agile/Scrum is suitable for this project because the system consists of several components that can be developed and tested incrementally, including the graph visualisation, classical solver, quantum execution engine and comparative analytics dashboard. The integration with IBM Quantum Cloud may also require adjustments during development due to testing results and external service limitations. Scrum will allow the team to divide the project into smaller iterations, regularly evaluate progress, identify problems early and improve the system based on feedback.

3.2. Process Elements

3.2.1. SE Principles

Modularity:
The system will be divided into independent modules including the graph management, classical solver, quantum execution engine and comparative analytics components. This will make the software easier to develop, test, maintain and modify.

Separation of Concerns:
Each component of the system will have a specific responsibility. For example, the user interface will handle user interaction and visualisation while the backend will handle problem processing and communication with IBM Quantum Cloud. This will reduce unnecessary dependencies between different parts of the system.

3.2.2. SE Methods/Techniques

Requirements Engineering:
The team will identify, document and continuously refine the functional and non functional requirements of the system. User stories will be used to describe the functionality from the user's perspective.

Software Testing:
Testing will be performed throughout development to ensure that individual components and the complete system operate correctly. Unit testing will be used for individual modules while integration testing will verify components such as quantum execution engine and results dashboard work correctly together.

3.2.3. SE Goals (ISO 25010)

Usability:
The application should provide a clear and intuitive interface that allows users to create graphs, execute Max Cut problems and understand the comparisons

between classical and quantum results without requiring extensive knowledge of quantum computing.

Performance Efficiency:
The system should process graph inputs efficiently and present results within a reasonable time. The application should also distinguish between the processing time of the classical approach and delays caused by IBM Quantum Cloud service.

3.2.4. SE Umbrella Activities

Software Configuration Management:
Git and Github will be used to manage different versions of the source code, track changes and support collaborative development between team members.

Software Quality Assurance:
The team will continuously monitor the quality of the software through testing, code reviews and verification of the system against its requirements. This will help ensure that the final system meets the expected functional and quality requirements.

3.3. Toolchain

A range of software engineering and quantum computing tools and technologies will be used to support the development and testing process throughout the project.

- **Version Control and Collaboration:**
  Git and GitHub will be used to manage the project's source code and allow team members to work together. They will also help track changes, manage different versions of the system and support collaboration between team members. GitHub Projects may be used to organise tasks and track progress during each sprint.

- **Development Environment:**
  Visual Studio Code will be used as the main development environment. Python will be the main programming language used to develop the Max-Cut algorithms and quantum computing features, while suitable web technologies will be used to create the user interface.

- **Quantum Computing Tools:**
  Qiskit and IBM Quantum Cloud Runtime will be used to implement and run the Quantum Approximate Optimisation Algorithm (QAOA). These tools will allow the system to execute quantum algorithms using IBM's quantum computing resources.

- **Graph Processing and Classical Algorithms:**
  NetworkX and NumPy will be used to create and process graphs, perform numerical calculations and support the implementation of classical Max-Cut algorithms.

- **Testing and Quality Assurance:**
  Testing will be performed throughout development to verify the correctness of individual components and the integrated system. GitHub will support code reviews and collaborative quality assurance, while a simulator will be used to test and validate quantum algorithms before execution on IBM Quantum hardware.

- **Continuous Integration and Documentation:**
  GitHub Actions may be used to automatically run tests and basic quality checks when changes are made to the project. GitHub will be used to maintain project documentation, including technical information, development notes and user guidance.

Overall, these tools will support the Agile/Scrum development process by helping the team collaborate, manage code, test the system and integrate the application with IBM's quantum computing services.

4. Project Scope & Requirements

4.1. Functional Requirements

- **Classical Solver Module:** Executes exact brute-force search (only for small *N*) and/or greedy/heuristic methods to compute maximum cut.
- **Quantum Execution Engine:** Converts the network graph into a format the quantum processor can understand, then uses IBM's cloud service to run a quantum optimization algorithm and return the best possible solution.
- **Comparative Analytics Dashboard:** Displays the side-by-side metrics of the classical computer and quantum processor, enabling the user to conveniently observe both approaches. Metrics like max cut weight achieved, total computation time, and probability distributions of sampled bitstrings. The scope can slightly be altered due its complexity and time.

4.2. Non-Functional Requirements

- **Reliability:** Graceful fallbacks to local Qiskit Aer simulators if IBM Quantum cloud queues exceed user-defined timeout limits.
- **Security:** Secure handling of the IBM Quantum API tokens. Either will be stored safely in a local browser storage or environment configuration.